like1000

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

This .tar file got tarred a lot.

💡 Hints

- Try and script this, it'll save you a lot of time

🧰 Tools Used

- `7z` – Extract compressed files
- `tar` – Extract tar archives
- **Bash** – Shell scripting for loop automation
- **Python** – Alternative scripting approach

🔍 Complete Solution

Step 1: Download and Prepare

Download the file named `1000.tar` from the challenge page.

Step 2: First Extraction

Extract the initial archive:

```
7z x 1000.tar
```

Output:

```
...
```

After extraction, you get two files:

- `999.tar`
- `filler.txt`

Step 3: Observe the Pattern

Extracting `999.tar` gives:

- `998.tar`
- `filler.txt`

This pattern continues: each `.tar` file contains the next numbered `.tar` file and a `filler.txt` file. The numbering goes down from `1000` to `1`.

Step 4: Manual Extraction (Too Slow)

Manually running `tar -xvf` for each file from 1000 down to 1 would be extremely tedious. Let's automate it!

Step 5: Automated Extraction with Bash Loop

Use a `for` loop to extract all archives sequentially:

```
for i in {1000..1}; do tar -xvf $i.tar; done
```

How it works:

- `{1000..1}` generates the sequence: 1000, 999, 998, ..., 1
- `tar -xvf $i.tar` extracts the archive named `$i.tar`

- The loop continues until all archives are extracted

Output (excerpt):

```
1000.tar
999.tar
filler.txt
998.tar
filler.txt
...
1.tar
filler.txt
flag.png
```

After the loop finishes, you'll have a file named `flag.png` !

Step 6: Alternative – Python Script

If you prefer Python:

```python
import tarfile
import os

i = 1000
while i > 0:
    filename = f"{i}.tar"
    if os.path.exists(filename):
        with tarfile.open(filename, 'r') as tar:
            tar.extractall()
        i -= 1
    else:
        break

print("Extraction complete!")
```

Step 7: View the Flag

Now open the extracted image:

```
xdg-open flag.png    # Linux
open flag.png        # macOS
start flag.png       # Windows
```

The flag is visible directly in the image!

📌 One-Liner Solution

You can combine everything into a single command:

```
for i in {1000..1}; do tar -xvf $i.tar 2>/dev/null; done; xdg-open flag.png
```

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, the image will reveal the complete flag.

🧠 Key Concepts

Concept	Description
Nested Archives	Archives inside archives – like Russian nesting dolls (Matryoshka).
Tar (Tape Archive)	A common archive format on Unix/Linux systems (`.tar`).
Bash Loops	Automating repetitive tasks with `for` loops in shell.
Sequential Extraction	Extracting archives in reverse order (from 1000 down to 1).
Automation	The key to solving challenges with many repetitive steps.

💡 Pro Tips

1. **Automate repetitive tasks** – If you find yourself doing the same command many times, write a loop or script.
2. **Bash `for` loops are powerful** – `for i in {1000..1}; do command; done` iterates from 1000 down to 1.
3. **Suppress output noise** – Use `2>/dev/null` to hide error messages (like when files already exist).
4. **Check the pattern first** – Before writing a script, understand the naming convention and extraction order.
5. **Use `ls` to verify** – After extraction, `ls` will show you the current files and the next archive to extract.
6. **Tar doesn't need `-v`** – The `-v` (verbose) flag shows output; omit it with `-xvf` for cleaner output.

🖇️ Alternative Automation Tools

Tool	Command / Approach
Bash Loop	`for i in {1000..1}; do tar -xvf $i.tar; done`
Python	`while i > 0: tarfile.open(f"{i}.tar").extractall(); i -= 1`
find + exec	`find . -name "*.tar" -exec tar -xvf {} \;` (but order matters!)